

Resolución de la Actividad 8: Clase 04

Grupo 4

Integrantes:

- Avila, Aldana, legajo 1193721, alavila@uade.edu.ar
- Caivano, Alexander Francisco, legajo 1209389, acaivano@uade.edu.ar
- Casareski, Juan Ignacio, legajo 1176109, jcasareski@uade.edu.ar
- Segura, Juan Ignacio, legajo 1242159, jsegura@uade.edu.ar

Parte A: Identificación de candidatos para Redis

Se evalúa cada dato de FlowOps como candidato a Redis. El criterio: Redis es la capa rápida y efímera, no la fuente de verdad. La definición versionada y el estado persistente viven en MongoDB; el registro de eventos vive en Cassandra.

Dato o función	¿Usar Redis?	Motivo	Riesgo
Estado actual de una instancia activa	Parcial	Lectura rápida del estado en curso. Es una copia, la verdad está en MongoDB.	Si Redis cae, se reconstruye desde MongoDB. No usarlo como única fuente.
Historial completo de eventos	No	Volumen append-only intensivo. Vive en Cassandra.	Saturaría la memoria y no es el patrón de Redis.
Definición publicada de un proceso	Parcial	Cache-aside para no leer MongoDB en cada inicio de instancia.	Definición desactualizada si no se invalida al publicar una versión nueva.
Sesión de usuario autenticado	Sí	Dato efímero con expiración natural por inactividad.	Pérdida de sesiones si Redis se reinicia sin persistencia.
Tareas pendientes por rol	Parcial	Índice rápido para la bandeja de trabajo de cada rol.	La verdad está en MongoDB. Se reconstruye si se pierde.
Contador de instancias iniciadas por día	No		
Rate limit por tenant	Sí	Ventana corta en memoria, no necesita persistir.	Si se pierde, se reinicia el conteo. Impacto bajo.
Cola de notificaciones pendientes	Sí	List eficiente para encolar y consumir pendientes.	Pérdida de pendientes si Redis cae sin AOF.
Métricas agregadas temporales	No		
Datos maestros del tenant	Parcial	Solo se cachea la configuración de lectura frecuente.	Fuente de verdad en MongoDB. Invalidar al cambiar la config.

Respuestas a las preguntas:

1. ¿Qué datos deberían estar siempre en una base persistente además de Redis?
Las definiciones de proceso, el estado de las instancias y los datos maestros del tenant. Todo lo que sea fuente de verdad.

2. ¿Qué datos podrían perderse sin comprometer la auditoría?
Sesiones, caché, rate limit y la cola en proceso. La auditoría vive en Cassandra, así que no depende de Redis.
3. ¿Qué datos deberían tener TTL?
Sesiones, caché de definición, rate limit y el estado de instancias ya finalizadas.
4. ¿Qué datos no deberían guardarse en Redis?
El historial de eventos y la auditoría, y cualquier dato como única copia de una definición o del estado de una instancia.

Parte B: Diseño de claves

Convención: recurso:{tenant_id}: ..., con prefijo por tipo de recurso y el tenant_id como segundo segmento para aislar datos por empresa. Segmentos en minúscula separados por dos puntos.

Uso	Clave propuesta	Ejemplo concreto	Motivo
Configuración de tenant	tenant:{tenant_id}:config	tenant:empresa_acme:config	Caché de config de lectura frecuente, una por tenant.
Sesión de usuario	session:{session_id}	session:sess_123	La sesión no depende del tenant para ubicarse, se busca por su id.
Estado actual de instancia	instance:{tenant_id}:{instance_id}:state	instance:empresa_acme:inst_1001:state	Acceso directo al estado de una instancia concreta.
Tareas pendientes por rol	tasks:{tenant_id}:{role}:pending	tasks:empresa_acme:compras:pending	Una cola por rol y tenant para armar la bandeja de trabajo.
Rate limit por tenant	rate_limit:{tenant_id}:{window}	rate_limit:empresa_acme:20260609T1030	Contador por ventana de tiempo, se descarta al pasar la ventana.
Cola de notificaciones	queue:{tenant_id}:notifications	queue:empresa_acme:notifications	Una cola de pendientes por tenant.
Cache de definición de proceso	process_def:{tenant_id}:{process_id}:{version}	process_def:empresa_acme:alta_proveedor:1	La versión en la clave evita servir una definición vieja por otra.

Parte C: Selección de estructuras Redis

Caso	Estructura Redis	Justificación	Operaciones principales
Sesión de usuario	Hash	Campos sueltos (user_id, tenant_id, role, last_activity) que se leen juntos y se pueden actualizar de a uno.	HSET, HGETALL, EXPIRE
Estado actual de instancia	Hash	Estado con campos (status, current_node, updated_at) que se actualizan parcialmente.	HSET, HGETALL, EXPIRE

Caso	Estructura Redis	Justificación	Operaciones principales
Tareas pendientes por rol	Sorted Set	Permite ordenar por SLA o prioridad y sacar primero la más urgente.	ZADD, ZRANGE, ZREM
Usuarios conectados por tenant	Set	Conjunto sin duplicados de usuarios activos. Alta y baja en O(1).	SADD, SREM, SCARD
Ranking de procesos más ejecutados	Sorted Set	El score ordena los procesos por cantidad de ejecuciones. Se actualiza desde el cálculo agregado.	ZINCRBY, ZREVRANGE
Cola de notificaciones	List	Encolar por un extremo y consumir por el otro, en orden de llegada.	LPUSH, RPOP, LLEN
Eventos recientes para monitoreo	Stream	Registro acotado de últimos eventos con id por entrada, ideal para leer en orden y limitar el tamaño.	XADD, XRANGE
Rate limit por tenant	String	Un contador entero por ventana, con incremento atómico.	INCR, EXPIRE

Ejemplo:

Caso: estado actual de instancia. Estructura: Hash. Clave: instance:empresa_acme:inst_1001:state. Campos: status, current_node, updated_at, assigned_role. Operaciones: HSET, HGETALL, EXPIRE.

Parte D: TTL y expiración

Clave o tipo de dato	¿Tiene TTL?	TTL propuesto	Justificación
Sesión de usuario	Sí	30 minutos, renovable	Caduca por inactividad. Cada actividad renueva el plazo.
Cache de proceso publicado	Sí	10 minutos	Plazo corto para que un cambio de definición se propague pronto.
Estado de instancia activa	Sí	24 horas, renovado mientras avanza	Evita claves muertas si la instancia queda colgada. Se renueva en cada paso.
Instancia finalizada	Sí	5 minutos	Ya está persistida en MongoDB. Se limpia rápido de memoria.
Rate limit	Sí	Igual a la ventana (por ejemplo 60 segundos)	La clave solo tiene sentido dentro de su ventana.
Cola de notificaciones	No	n/a	Vive hasta que se consume. No se descarta por tiempo.

Preguntas guía:

1. ¿Qué ocurre si una clave expira demasiado rápido?
Se pierde el dato antes de tiempo: una sesión se cierra sola o se fuerza un cache miss extra contra la base persistente.
2. ¿Qué ocurre si nunca expira?
Fuga de memoria. Se acumulan claves muertas (sesiones viejas, estados de instancias terminadas) que nunca se liberan.
3. ¿Qué datos requieren invalidación manual?
El caché de definición de proceso. Al publicar una versión nueva hay que borrar la clave vieja (DEL), no esperar al TTL.
4. ¿Qué datos pueden regenerarse desde MongoDB o Cassandra?
El caché de definición y el estado de instancia desde MongoDB. El ranking y las métricas desde el registro de eventos en Cassandra.
5. ¿Qué datos no deberían depender exclusivamente de Redis?
El estado de las instancias y las definiciones de proceso. Nada que sea fuente de verdad.

Parte E: Patrones de uso

Cache-aside para definición de proceso

Flujo para obtener la definición publicada de un proceso:

1. La API recibe una solicitud para iniciar una instancia.
2. Busca en Redis la clave `process_def:{tenant_id}:{process_id}:published`.
3. Si existe, usa la definición cacheada.
4. Si no existe:
 - Consulta la definición en MongoDB.
 - La guarda en Redis con TTL de 10 minutos.
 - Usa la definición recuperada.
5. Si el proceso cambia, se debe invalidar la clave en Redis (DEL) al publicar la versión nueva, para no servir una definición vieja.

Riesgo de una definición desactualizada: una instancia nueva podría arrancar con pasos o reglas de una versión anterior. El TTL corto acota la ventana y la invalidación manual al publicar la cierra.

Sesiones de usuario

Estructura de una sesión:

```
{
  "user_id": "usuario_123",
  "tenant_id": "empresa_acme",
  "role": "compras",
  "created_at": "2026-06-09T10:00:00Z",
  "last_activity": "2026-06-09T10:25:00Z"
}
```

Decisión	Respuesta
Clave Redis	<code>session:{session_id}</code>
Estructura	Hash
TTL	30 minutos, renovable por actividad
Operación para crear	HSET de los campos y EXPIRE
Operación para renovar	EXPIRE al detectar actividad
Operación para cerrar sesión	DEL

Decisión	Respuesta
Riesgo principal	Si Redis se reinicia sin persistencia, se caen las sesiones activas y los usuarios deben volver a autenticarse.

Tareas pendientes por rol

Las tareas pendientes son la bandeja de trabajo de un rol. Cuando una instancia llega a un paso humano (por ejemplo `revisión_compras` asignado al rol `compras`), queda esperando que alguien de ese rol la tome. Se guardan en una clave por rol y tenant, `tasks:{tenant_id}:{role}:pending`.

Respuestas:

1. ¿Usarían List, Set o Sorted Set?
Sorted Set, para ordenar por SLA o prioridad y atender primero lo más urgente.
2. ¿Necesitan conservar orden?
Sí, el orden de atención importa. El score del Sorted Set lo define.
3. ¿Necesitan prioridad?
Sí, una tarea próxima a vencer su SLA debe salir antes. Va en el score.
4. ¿Una tarea puede estar duplicada?
No. El Sorted Set evita duplicados por miembro (el id de tarea).
5. ¿Qué pasa si Redis se reinicia?
Se reconstruye la bandeja desde MongoDB, que tiene el estado real de cada instancia y su tarea pendiente.
6. ¿Dónde queda la versión persistente de la tarea?
En MongoDB, embebida en la instancia como `current_task`.

Rate limit por tenant

Rate limit simple para evitar que un tenant abuse de la API, con una clave por ventana de tiempo: `rate_limit:{tenant_id}:{window}`.

Parámetro	Valor propuesto
Ventana de tiempo	1 minuto
Máximo de solicitudes	100 por ventana
Estructura Redis	String contador
Operación principal	INCR sobre la clave de la ventana
TTL	60 segundos, igual a la ventana
Respuesta si supera el límite	Rechazar con HTTP 429 hasta la próxima ventana

Parte F: Comandos Redis esperados

Caso	Comando Redis propuesto	Explicación
Crear sesión	<code>SET session:sess_123 '{ "user_id": "u1", "tenant_id": "acme", "role": "compras" }'</code> <code>EX 1800</code>	Crea la sesión con expiración de 30 minutos.

Caso	Comando Redis propuesto	Explicación
Renovar sesión	EXPIRE session:sess_123 1800	Reinicia el plazo de inactividad al detectar actividad.
Guardar estado actual	HSET instance:acme:inst_1001:state status "waiting_task" current_node "revision_compras"	Actualiza los campos del estado de la instancia.
Agregar tarea pendiente	ZADD tasks:acme:compras:pending 1718000000 task_9001	Encola la tarea con su SLA como score para ordenar.
Completar tarea pendiente	ZREM tasks:acme:compras:pending task_9001	Saca la tarea de la bandeja del rol.
Aplicar rate limit	INCR rate_limit:acme:20260609T1030	Suma una solicitud en la ventana actual. Si pasa el máximo, se rechaza.
Encolar notificación	LPUSH queue:acme:notifications '{ "to": "compras", "type": "new_task" }'	Agrega una notificación al frente de la cola.

Parte G: Riesgos de Redis

Pérdida de datos en memoria

1. ¿Qué datos no deberían depender exclusivamente de Redis?
El estado de las instancias, las definiciones de proceso y los datos maestros del tenant.
2. ¿Qué datos pueden reconstruirse?
El caché de definición y el estado de instancia desde MongoDB. El ranking desde el registro de eventos en Cassandra.
3. ¿Qué persistencia configurarían conceptualmente: RDB, AOF o ninguna?
AOF para las sesiones y la cola de notificaciones, donde perder lo último escrito molesta. Para el caché alcanza con ninguna o un RDB esporádico, porque se regenera.
4. ¿Qué impacto tendría perder sesiones activas?
Los usuarios conectados se desloguean y deben autenticarse de nuevo. No se pierde información de negocio, solo continuidad de la sesión.

Datos desactualizados en caché

1. ¿Qué pasa si cambia una definición de proceso y Redis mantiene una versión vieja?
Una instancia nueva podría arrancar con pasos o reglas de una versión anterior.
2. ¿Qué estrategia usarían: TTL corto, invalidación manual o versionado?
Las tres combinadas: la versión en la clave evita colisiones, el TTL corto acota la ventana y la invalidación manual (DEL) al publicar la cierra.
3. ¿Cómo lo relacionan con consistencia eventual?
Durante el TTL, Redis y MongoDB pueden diferir. El sistema tolera ese desfase corto y converge cuando expira o se invalida la clave.

Uso excesivo de memoria

1. ¿Qué claves podrían crecer sin control?
La cola de notificaciones si no se consume, y los estados de instancia si no expiran al finalizar.
2. ¿Qué TTL o límite aplicarían?
TTL en sesiones, caché y estados. Para el Stream de monitoreo, un MAXLEN que recorte las entradas viejas.

3. ¿Qué política de eviction sería razonable?
volatile-lru: expulsar solo claves con TTL y menos usadas, sin tocar las que no deben perderse.
4. ¿Qué datos nunca deberían ser expulsados sin respaldo persistente?
Cualquier dato sin copia en MongoDB o Cassandra. Con volatile-lru, las claves sin TTL quedan a salvo de la expulsión.

Parte H: Integración con la arquitectura políglota

Subsistema de FlowOps	Base principal	¿Redis interviene?	Uso de Redis
Definiciones de procesos	MongoDB	Sí	Caché cache-aside con TTL.
Estado actual de instancia	MongoDB	Sí	Hash duplicado para lectura rápida.
Eventos de auditoría	Cassandra	No	
Tareas humanas	MongoDB	Sí	Índice de pendientes por rol.
Sesiones	Redis	Sí	Almacenamiento principal efímero con TTL.
Métricas temporales	Cassandra	No	
Notificaciones	Redis	Sí	Cola List de pendientes.
Validación de flujo	MongoDB	No	